Stress + Chaos Test Strategy

Stress + Chaos Test Strategy

Revision: 1 **Last modified:** 2026-06-26T12:00:00Z

> Master technical specification — Volume 8 (Testing & QA), nano-detail document **stress-chaos**, one of the seven §11.4.169 cross-cutting test-type deep-dives. It deepens `10-testing-acceptance-and-qa.md` §5.9 (STRESS + CHAOS) into an implementation-ready bank mapped onto the §11.4.85 **closed-set** helper contracts (`ab_stress_run`, `ab_chaos_kill_pid_during`, …) with chaos-injection cleanup in `trap EXIT` (§11.4.14) and target/hardware safety (§11.4.133). SPEC-ONLY: it describes harness, fixtures, evidence, gate, and the paired §1.1 mutation; it does not build the product. Sources cited inline by id — `[OVERVIEW]` = doc 10; `[TM]` = `[../v05-security/threat-model.md]`; `[reconcile]` = `[../v03-control-plane/reconciliation-flow.md]`; `[ladder]` = `[../v02-data-plane/transport-selection-ladder.md]`; `[01-DP]` = doc 01. Claims not grounded in the evidence base are marked **UNVERIFIED** per constitution §11.4.6 — never fabricated.

Table of contents

---

0. Scope on HelixVPN surfaces

§11.4.85 makes resilience a per-fix obligation: happy-path coverage alone is a §11.4 bluff at the resilience layer. For a VPN the resilience failures are the *dangerous* ones — a tunnel that drops mid-transfer must **not leak plaintext during the gap**; a reconcile that races a Redis drop must **not lose a delta**; a torn network-map snapshot must **never** be applied half-written. This bank exercises every HelixVPN fix under sustained load, concurrent contention, boundary inputs, and failure injection, and asserts categorised recovery with captured evidence.

Surface	STRESS dimension	CHAOS injection	Recovery invariant
coordinator reconcile	≥100 policy edits / ≥30 s churn	Redis drop mid-reconcile	no lost delta; degrades gracefully ([reconcile §0.2])
edge forwarding	sustained `iperf3` transfer	SIGKILL the edge mid-transfer	tunnel re-establishes; **zero leak in the gap** (S9)
client transport ladder	repeated re-pick under flap	interface flap mid-flow	re-establishes < 3 s; kill-switch sealed during churn ([ladder §3])
coordinator fan-out	≥10 concurrent enrollments	partial-write of map snapshot	recovery restores a consistent (not torn) map
connector liveness	N connectors attach/detach	connector process kill	peer drops from every map within convergence budget
IPAM allocator	boundary ranges (empty/max/off-by-one)	mid-write SIGKILL of the alloc tx	no double-allocation; tx rolls back atomically

1. The §11.4.85 closed-set, instantiated for HelixVPN

§11.4.85 fixes a closed, mechanically-auditable set. Each member is instantiated below; the generic `ab_*` helper is in column 3.

STRESS (closed-set):

§ member	HelixVPN instantiation	Helper
sustained load (N≥100 OR ≥30 s, p50/p95/p99 recorded)	≥100 `policy apply` edits driving reconcile; ≥30 s `iperf3` through the edge	`ab_stress_run`
concurrent contention (N≥10 parallel, no deadlock/leak)	≥10 concurrent enrollments; ≥10 concurrent `WatchNetworkMap` opens	`ab_stress_concurrent`
		`ab_stress_boundary`

§ member	HelixVPN instantiation	Helper
boundary conditions (empty/max/off-by-one)	empty ACL; max-peer NetworkMap; off-by-one overlay `/48` range end	

CHAOS (closed-set, applied per fix-class appropriateness):

§ member	HelixVPN instantiation	Helper
process-death injection	SIGKILL the edge / coordinator / connector mid-call	`ab_chaos_kill_pid_during`
network-fault injection	drop/delay/reorder via `tc netem`; Redis Streams partition	`ab_chaos_drop_network_during`
input-corruption injection	corrupt the map-snapshot file / `.env` mid-test	`ab_chaos_corrupt_file_during`
resource-exhaustion injection	edge OOM pressure; coordinator FD exhaustion; disk-full on the DB volume	`ab_chaos_oom_pressure_during` / `ab_chaos_disk_full_during`
state-corruption injection	partial-write of the network-map snapshot; mid-flight Redis lock loss	`ab_chaos_corrupt_file_during` (+ recovery assert)

2. Harness — the §11.4.85 helper contracts

The bank is built **only** on the §11.4.85 `stress_chaos.sh` helper library; each helper composes with `ab_pass_with_evidence` / `ab_skip_with_reason` (§11.4.69) and a mandatory `trap '...' EXIT` cleanup (§11.4.14). The helpers are run via `make chaos` (overview §9), backgrounded (§11.4.89) so the main work stream is never blocked, and dispatched by `helix_qa` under the real-time sync channel (§11.4.116).

```
# stress_chaos.sh contract surface (§11.4.85) — the only API this
        bank calls
ab_stress_run        <name> <N|secs> <fn>          # sustained;
        records latency.json p50/p95/p99
ab_stress_concurrent <name> <N> <fn>               # N parallel;
        asserts no deadlock/leak
ab_chaos_kill_pid_during   <pid> <workload> --on-kill <hook> --
        assert <cond> --evidence <dir>
ab_chaos_drop_network_during <iface|stream> <workload> --assert
        <cond> --evidence <dir>
ab_chaos_corrupt_file_during <path> <workload> --assert <cond> --
        evidence <dir>
```

```
ab_chaos_oom_pressure_during <pid> <workload> --assert <cond> --
        evidence <dir>
ab_chaos_disk_full_during   <mount> <workload> --assert <cond> --
        evidence <dir>

# CHAOS: kill the edge mid-transfer, assert recovery AND no leak
        during the gap (overview §5.9)
ab_chaos_kill_pid_during "$(edge_pid)" 'ip netns exec client
        iperf3 -t 20 -c 10.10.0.20' \
  --on-kill 'sleep 3' \
  --assert  'tunnel_reestablished_within 3s &&
        killswitch_sealed_during_gap' \
  --evidence "qa-results/chaos/edge_kill_$(date +%s)/"
# the assertion re-reads the gap pcap: ZERO plaintext egress while
        the edge was dead (composes S4/S9)
```

Single-resource-owner partitioning (§11.4.119): exactly one process owns the edge data port during a chaos run; concurrent observers are read-only probes. Each chaos run owns its rig exclusively so evidence is never cross-contaminated.

---

3. Target/hardware safety (§11.4.133) + cleanup (§11.4.14)

Chaos injection on a VPN must be **safe for the target system and its host** — §11.4.133 forbids any unsafe write. HelixVPN's injections are all reversible, bounded, software-only:

- **No unsafe hardware/system writes.** Injections touch *process* and *namespace* state only — `kill`, `tc netem`, a corrupt copy of a *test* snapshot file, `cgroup` memory pressure on the *test* edge container, a `tmpfs`-bounded disk-full on a *scratch* mount. No kernel/voltage/clock register, no real partition, no host firewall outside the netns table (§11.4.133 (2)).
- **Bounded blast radius (§12 host-safety).** OOM pressure is applied via the test container's cgroup `MemoryMax`, never the host's; disk-full uses a small `tmpfs` scratch volume, never the host root; the edge runs rootless (§11.4.161) so a chaos-induced crash cannot escalate.
- **Cleanup is non-negotiable (§11.4.14).** Every helper installs `trap '<cleanup>' EXIT`: corrupt-restore (restore the snapshot from a pre-op copy, §9.2), disk-fill-cleanup (remove the `tmpfs` filler), process-restart (the supervisor brings the killed process back), `tc qdisc del` (remove netem), netns teardown. A cleanup failure is itself a §11.4.14 violation — the orchestrator runs a post-test quiescence check and FAILs the just-completed run if it left orphan state.

```
# every chaos helper's mandatory cleanup shape (§11.4.14 +
        §11.4.133)
trap 'restore_snapshot_from_backup; rm -f "$TMPFS_FILLER"; tc
        qdisc del dev "$IFACE" root 2>/dev/null;
      supervisor_restart "$KILLED"; rig/netns_down.sh' EXIT
```

---

4. Fixtures — real (§11.4.27)

Stress/chaos is below the unit layer: **no mocks**. The system under chaos is the real, fully-wired stack booted via `containers` (§11.4.76).

Fixture	What	Why real
`real_edge` + `real_coordinator` + real PG/Redis	the wired stack via `containers/pkg/boot`	chaos must hit real recovery paths, not stubbed ones
`policy_churn.yaml` (≥100 edits)	a real sequence of `policy apply` commands	sustained reconcile load must be real compile+stream work
`map_snapshot.bin` + `map_snapshot.bin.bak`	a real network-map snapshot + a pre-op backup	the corruption target + the §9.2 restore source
`connectors_10.toml`	10 real connector configs	concurrent attach/detach must be real enrollments

5. Captured evidence (§11.4.69 / .85)

Every PASS cites the §11.4.85-mandated artifact set under `qa-results/{stress,chaos}/<run-id>/`:

Artifact	From	Asserts
`latency.json`	`ab_stress_run`	per-iteration p50/p95/p99 within budget under sustained load
`categorised_errors.txt`	every helper	per-error-class counts (refused-cleanly vs crashed) — §11.4.85
`recovery_trace.log`	`ab_chaos_*`	the recovery timeline (kill→detect→re-establish)
`state_delta_snapshot.json`	state-corruption runs	the post-recovery state == a consistent state (not torn)
`gap_pcap.pcap`	edge-kill / iface-flap runs	**zero** plaintext / `:53` during the recovery gap (S9)
`concurrency_result.json`	`ab_stress_concurrent`	N parallel ops, zero deadlock, zero resource leak

Anti-bluff (overview §0): a chaos PASS that only proves "the process was killed" is a B-class bluff — it MUST prove *categorised recovery* (the `recovery_trace.log` + the no-leak `gap_pcap`), or it FAILs.

---

6. Determinism (§11.4.50)

Chaos is timing-sensitive, so determinism is asserted on the **recovery verdict**, not the raw timing. `ab_run_n_times "chaos-edge-kill" 3 run_edge_kill` runs N=3 against the same artifact MD5; the evidence-hash is over the verdict tuple `(reestablished_within_budget: bool, leak_during_gap: bool, lost_delta: bool)`; all three runs MUST yield the identical tuple. A run where the tunnel re-establishes in 2 of 3 is **auto-FAIL** (§11.4.50, no flake escape). The raw kill→recover timings are recorded for trend (§11.4.24). Cycle-validation runs N=10 for the leak-critical chaos cases.

---

7. Acceptance gate

Gate	Bar	Evidence	Phase
SC-EDGE-KILL	tunnel re-establishes ≤ budget; **zero** leak in the gap	`recovery_trace.log` + `gap_pcap.pcap`	MVP (release-blocking)
SC-REDIS-DROP	reconcile degrades gracefully; **no** lost delta after recovery	`state_delta_snapshot.json`	MVP
SC-IFACE-FLAP	client re-picks transport ≤ 3 s; kill-switch sealed during churn	`gap_pcap.pcap` + ladder telemetry	MVP
SC-SNAPSHOT-TORN	partial-write recovery yields a consistent map, never torn	`state_delta_snapshot.json`	MVP
SC-RECONCILE-CHURN	≥100 edits / ≥30 s: p99 reconcile within SLO; no leak/crash	`latency.json`	MVP
SC-ENROLL-CONCURRENT	≥10 concurrent enrollments: zero deadlock,	`concurrency_result.json`	MVP

Gate	Bar	Evidence	Phase
	zero double-alloc		
SC-OOM / SC-DISK-FULL	edge refuses cleanly OR degrades; **never** crashes	`categorised_errors.txt`	MVP

Every closed defect registers a **permanent CHAOS guard** (§11.4.135): the chaos test that reproduced the failure becomes a standing regression guard in `make chaos`. The leak-critical SC cases (SC-EDGE-KILL, SC-IFACE-FLAP) are §11.4.132 risk-ordered first.

---

8. The paired §1.1 mutation

```
MUTATION (paired §1.1, gate CM-CHAOS-NO-LEAK-DURING-GAP):
  Remove the kill-switch re-enforcement that runs BEFORE the FSM
  transition on tunnel drop (S9) — i.e. let the firewall briefly
open
  during the edge-kill recovery gap.
EXPECTED:  gap_pcap.pcap now shows plaintext (and/or :53) egress
during
           the gap → SC-EDGE-KILL FAILs → mutation caught.
RESTORE:   re-instate the pre-transition re-enforcement; re-run →
GREEN.
```

A second mutation (`CM-CHAOS-RECOVERY-RESTORES-CONSISTENT`) strips the snapshot-write atomicity so a corrupt/torn map is applied after a partial-write injection; expected: `state_delta_snapshot.json` shows a torn map → SC-SNAPSHOT-TORN FAILs → caught. A third (`CM-CHAOS-CLEANUP-PRESENT`) removes a helper's `trap EXIT` and asserts the orphan-state quiescence check FAILs (§11.4.14). All mutations restored and the tree verified quiescent (§11.4.84) before commit.

---

9. Test skeletons

```
# scripts/stress_chaos.sh — SC-REDIS-DROP: Redis drop mid-
        reconcile, no lost delta (§11.4.85)
set -euo pipefail
out="qa-results/chaos/redis_drop_$(date +%s)"; mkdir -p "$out"
trap 'restore_redis; rig/netns_down.sh'
        EXIT                          # §11.4.14
apply_policy_edits 120
        &                                                 # ≥100
        edits churning reconcile
PID_REDIS=$(redis_pid)
ab_chaos_drop_network_during "$PID_REDIS"
        'wait_for_reconcile_quiescent' \
```

```
  --assert 'all_deltas_delivered_after_recovery && no_torn_map' \
  --evidence "$out"
# the assertion diffs the post-recovery served maps against the
        authoritative CompiledPolicy:
# every affected device's final map MUST equal the compiled target
        (no lost delta).

# SC-RECONCILE-CHURN: ≥100 edits / ≥30 s sustained, p99 within SLO
        (§11.4.85 sustained-load)
ab_stress_run "reconcile-churn" 30
        'apply_one_policy_edit_and_wait_reconciled' \
  && ab_pass_with_evidence "reconcile p99 within SLO under 30s
        churn" "qa-results/stress/reconcile-churn/latency.json" \
  || ab_fail "reconcile degraded under sustained churn"

# SC-ENROLL-CONCURRENT: ≥10 parallel enrollments, no deadlock / no
        double-allocation (§11.4.85 concurrent)
ab_stress_concurrent "enroll-10" 10 'enroll_one_device' \
  --assert 'distinct_overlay_ips == 10 && deadlocks == 0' \
  --evidence "qa-results/stress/enroll-10/concurrency_result.json"

# SC-OOM: edge under memory pressure — refuses cleanly, never
        crashes (target-safe, §11.4.133)
ab_chaos_oom_pressure_during "$(edge_pid)" 'ip netns exec client
        iperf3 -t 10 -c 10.10.0.20' \
  --assert 'edge_refused_cleanly_or_degraded &&
        helix_edge_up_recovered' \
  --evidence "qa-results/chaos/edge_oom_$(date +%s)/"
# pressure applied via the test container cgroup MemoryMax — NEVER
        the host (§12 host-safety)
```

Honest boundary (§11.4.6). This bank proves *categorised, bounded* recovery for the modelled fault set; it does **not** prove recovery under faults outside the closed set (e.g. simultaneous multi-fault, byzantine clock skew) — those are stated coverage gaps, re-armed by §11.4.118 discovery passes. The concrete recovery budgets (3 s flap, edge restart budget) are MVP **measured** targets, **UNVERIFIED** until the rig produces the `recovery_trace.log`. The "no lost delta under Redis drop" claim rests on the Redis Streams consumer-group at-least-once semantics ([reconcile §0.3 `events:*` consumer group]) and is **UNVERIFIED** for the specific drop-timing until the chaos run executes.

Sources verified

- [OVERVIEW] [`../10-testing-acceptance-and-qa.md`] — §0 (bluff classes), §5.9 (STRESS+CHAOS strategy + helper list + edge-kill skeleton), §6 (coverage ledger), §8 (determinism/risk-order), §9 (`make chaos`, backgrounded), §11.4.135 regression guard. (Read 2026-06-26.)
- [TM] [`../v05-security/threat-model.md`] — `T-CLI-T-1/T-DP-D-1` (kill-switch leak window), S9 state machine, §10 R-RACE. (Read 2026-06-26.)
- [reconcile] [`../v03-control-plane/reconciliation-flow.md`] — §0.2 signature flow, §0.3 `events:policy/routes/devices` Redis Streams consumer group `coordinator`. (Read 2026-06-26.)

- [ladder] [`../v02-data-plane/transport-selection-ladder.md`] — §3 ladder state machine + flap re-pick budget. (Read 2026-06-26.)
- [01-DP] `final/01-data-plane.md` — edge fail-static (I3), WG core. (Cited via overview.)
- Constitution: §11.4.169, §11.4.85 (stress+chaos closed-set + helper contracts), §11.4.14 (cleanup/quiescence), §11.4.133 (target/hardware safety), §11.4.69 (sink-side evidence), §11.4.50 (determinism), §11.4.119 (single-resource-owner), §11.4.135 (regression guard), §11.4.89 (background execution), §11.4.116 (sync channel), §11.4.84 (quiescence), §12 (host safety), §9.2 (pre-op backup), §1.1 (paired mutation).